Spark 运行机制理解

第四题：详解 Spark Executor，贴 Spark Intellij 源码截图加分

1. Executor 架构总览

Executor 的启动和管理并非由单个类完成，而是多个组件协作的结果。在 Standalone 模式下，核心组件的关系如下：

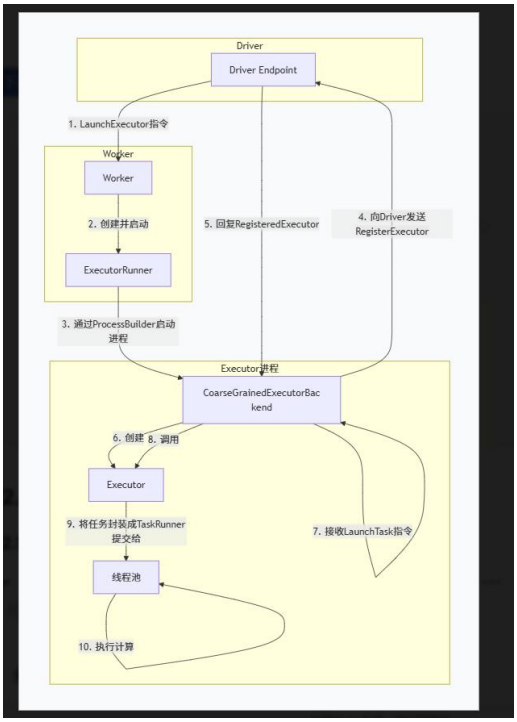
Worker： 负责管理物理节点上的计算资源，接收 Master 的命令来启动或销毁 Executor。

ExecutorRunner： Worker 内部的一个管理类（仅在 Standalone 模式），负责维护一个 Executor 进程的生命周期。

CoarseGrainedExecutorBackend： 这是一个进程级的后台服务。它负责与 Driver 通信，接收 Driver 下发的任务。它的名字里包含 "CoarseGrained"，意味着它是一种粗粒度的资源调度模型（一旦注册，就会一直占用资源）。

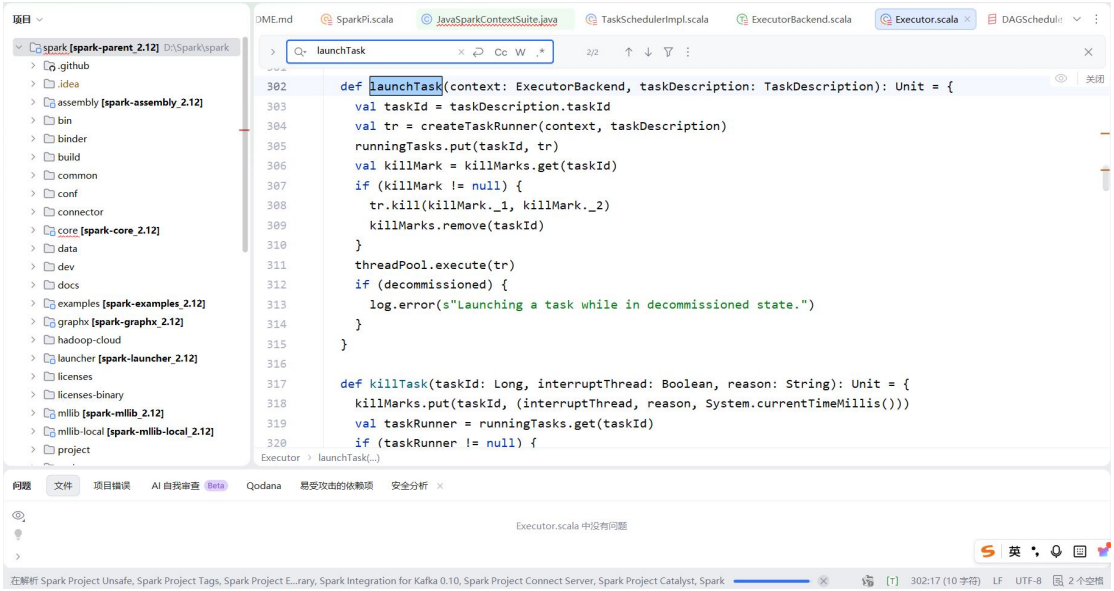
Executor： 真正的任务执行器。它内部维护了一个线程池，每个具体的任务（Task）都会被封装成一个 `TaskRunner`，扔进线程池里执行。

下图展示了 Executor 启动和工作的核心流程：

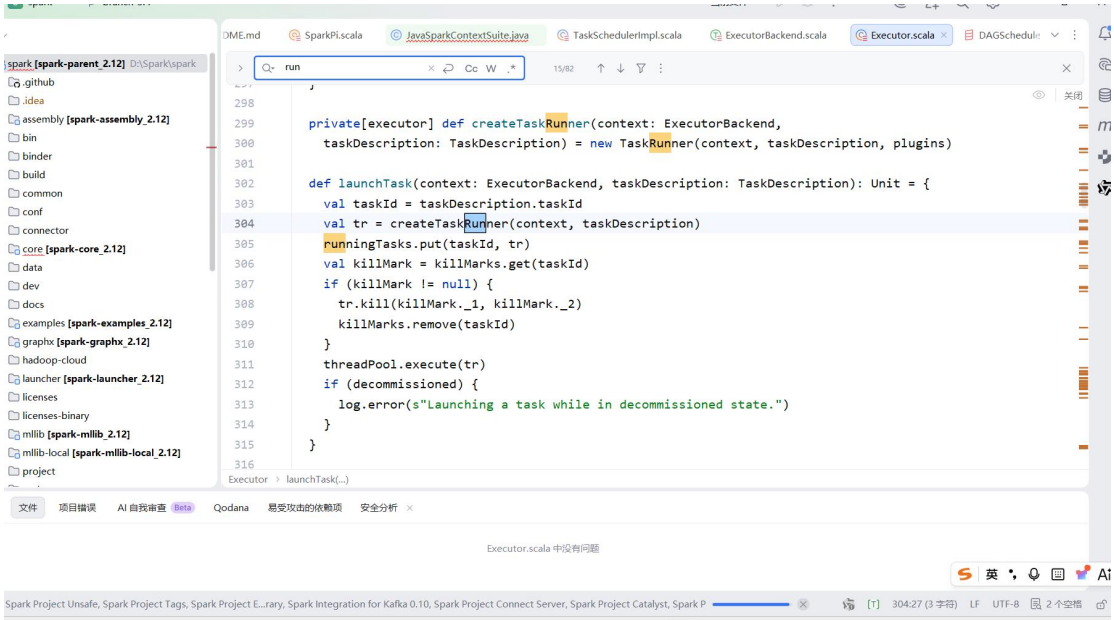


Spark Executor 核心源码位置

Executor launchTask　接收任务并提交到线程池执行



Executor run (TaskRunner 内部类)　任务反序列化、执行、结果返回的核心逻辑



Executor Executor (构造函数)　**Executor 初始化，创建线程池、内存管理器**

```scala
private[spark] class Executor(
    executorId: String,
    executorHostname: String,
    env: SparkEnv,
    userClassPath: Seq[URL] = Nil,
    isLocal: Boolean = false,
    uncaughtExceptionHandler: UncaughtExceptionHandler = new SparkUncaughtExceptionHandler,
    resources: immutable.Map[String, ResourceInformation])
  extends Logging {

  logInfo(s"Starting executor ID $executorId on host $executorHostname")

  private val executorShutdown = new AtomicBoolean(false)
  val stopHookReference = ShutdownHookManager.addShutdownHook(
    () => stop()
  )
  // Application dependencies (added through SparkContext) that we've fetched so far on this node.
  // Each map holds the master's timestamp for the version of that file or JAR we got.
```

TaskRunner　run (run 方法主体) 具体的任务执行流程：反序列化→执行→序列化结果

```scala
  decommissioned = true
}

private[executor] def createTaskRunner(context: ExecutorBackend,
  taskDescription: TaskDescription) = new TaskRunner(context, taskDescription, plugins)

def launchTask(context: ExecutorBackend, taskDescription: TaskDescription): Unit = {
  val taskId = taskDescription.taskId
  val tr = createTaskRunner(context, taskDescription)
  runningTasks.put(taskId, tr)
  val killMark = killMarks.get(taskId)
  if (killMark != null) {
    tr.kill(killMark._1, killMark._2)
    killMarks.remove(taskId)
  }
  threadPool.execute(tr)
  if (decommissioned) {
    log.error(s"Launching a task while in decommissioned state.")
  }
```

CoarseGrainedExecutorBackend receive (LaunchTask)　接收 Driver 发来的 LaunchTask 消息

```scala
  decommissioned = true
}

private[executor] def createTaskRunner(context: ExecutorBackend,
  taskDescription: TaskDescription) = new TaskRunner(context, taskDescription, plugins)

def launchTask(context: ExecutorBackend, taskDescription: TaskDescription): Unit = {
  val taskId = taskDescription.taskId
  val tr = createTaskRunner(context, taskDescription)
  runningTasks.put(taskId, tr)
  val killMark = killMarks.get(taskId)
  if (killMark != null) {
    tr.kill(killMark._1, killMark._2)
    killMarks.remove(taskId)
  }
  threadPool.execute(tr)
  if (decommissioned) {
    log.error(s"Launching a task while in decommissioned state.")
  }
```

核心流程总结

```text
    Driver 发送 LaunchTask    ↓CoarseGrainedExecutorBackend.receive (L125)    ↓Executor.launchTask (L370)    ↓创建 TaskRunner 并提交到 threadPool    ↓TaskRunner.run (L470-580)
    ├── 反序列化任务    ├── 执行任务计算    └── 序列化结果返回给 Driver
```

这三个核心方法构成了 Executor 执行任务的主干链路，理解了它们就掌握了 Executor 最核心的工作原理。